

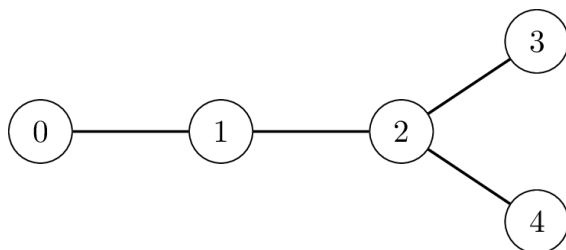
遷移

自然歷史博物館正在調查玻利維亞恐龍的遷移模式。古生物學家在 N 不同的地點發現了恐龍腳印，從 0 到 $N - 1$ ，依年齡遞減的順序排列：0 地點有最古老的腳印，而 $N - 1$ 地點則有最年輕的腳印。

恐龍從一個特定的、較舊的地點遷移到每個地點（地點 0 除外）。對於每個地點 i ，如 $1 \leq i \leq N - 1$ ，正好存在一個較舊的地點 $P[i]$ （與 $P[i] < i$ ），使一些恐龍直接從地點 $P[i]$ 遷徙到地點 i 。單一較舊的地點可能是遷移到多個較新地點的地點。

古生物學家將每次遷移建立為地點 i 與 $P[i]$ 之間的**非定向連接**。請注意，對於任何兩個不同的地點 x 和 y ，都有可能通過穿越連線序列從 x 遷移到 y 。個地點 x 和 y 之間的**距離**定義為從 x 到 y 所需的最少連接數。

例如，下圖顯示了 $N = 5$ 個地點的情況，其地點 $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, 和 $P[4] = 2$ 。人們可以透過 2 條連線從地點 3 到地點 4，因此它們之間的距離是 2。



博物館的目標是以最大可能的距離找出一對地點。

請注意，這一對不一定是唯一的：例如，在上面的例子中， $(0, 3)$ 和 $(0, 4)$ 這兩對的距離都是 3，這是最大值。在這種情況下，任何達到最大距離的資料對都被視為有效。

最初， $P[i]$ 的值是**不知道的**。博物館派遣研究小組依序造訪 $1, 2, \dots, N - 1$ 地點。到達每個地點 i ($1 \leq i \leq N - 1$) 之後，研究團隊會執行下列兩個動作：

- 確定 $P[i]$ 的值，也就是遷移到地點 i 的來源。
- 根據先前收集到的資訊，決定是否要傳送**一個**訊息到博物館，或是不傳送任何來自此地點的訊息。

訊息是透過昂貴的衛星通訊系統傳輸，因此每條訊息都必須是 1 和 20 000（含）之間的整數。此外，團隊最多只允許傳送**50 個訊息**。

您的任務是執行一項策略，藉此

- 研究團隊選擇傳送訊息的地點，以及每個訊息的價值。
- 博物館可以僅依據從每個地點收到的訊息，決定距離最遠的一對地點，並知道這些訊息是從哪些地點傳送的。

透過衛星傳送大量訊息的成本很高。您的得分將取決於傳送的最高整數和傳送的訊息總數。

實作細節

您需要執行兩個子程序；一個是研究團隊的子程序，另一個是博物館的子程序。

您應該為**研究團隊**執行的子程序是：

```
int send_message(int N, int i, int Pi)
```

- N ：有足跡的地點數量。
- i ：小組目前瀏覽的地點索引。
- P_i ： $P[i]$ 的值。
- 每個測試案例都會呼叫這個子程序 $N - 1$ 次，順序為 $i = 1, 2, \dots, N - 1$ 。

這個子程序應該回傳 $S[i]$ ，指定研究團隊在地點 i 所採取的行動：

- $S[i] = 0$ ：研究小組決定不從地點 i 傳送訊息。
- $1 \leq S[i] \leq 20\,000$ ：研究小組發送整數 $S[i]$ 作為來自地點 i 的訊息。

您應該為**博物館**實現的子程序是

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S ：一個長度為 N 的陣列，使得：
 - $S[0] = 0$.
 - 對於每個 $1 \leq i \leq N - 1$, $S[i]$ 是 `send_message(N, i, Pi)` 所傳回的整數。
- 對於每個測試案例，這個子程序都會被呼叫一次。

此子程序應傳回距離最大的一對地點 (U, V) 。

在實際評分中，調用上述子程序的程式會運行**正好兩次**。

- 在程式第一次執行時
 - `send_message` 恰好被呼叫 $N - 1$ 次。
 - 您的程式可以在連續的呼叫中儲存和保留資訊。
 - 返回值（陣列 S ）由評分系統儲存。
 - 在某些情況下，分級系統的行為是**適應性的**。這表示 $P[i]$ 在呼叫 `send_message` 的值可能取決於研究團隊在之前的呼叫中所採取的行動。
- 在程式的第二次執行過程中：
 - `longest_path` 正好被呼叫一次。`longest_path` 在第一次執行時唯一可用的資訊是陣列 S 。

約束

- $N = 10\,000$
- $0 \leq P[i] < i$ 對於每個 i ，使得 $1 \leq i \leq N - 1$.

子任務及評分

子任務	評分	額外的限制條件
1	30	地點 0 和其他一些地點在所有地點對中有最大距離。
2	70	無額外限制條件。

讓 Z 成為 S 陣列中出現的最大整數，讓 M 成為研究團隊傳送的訊息數量。

在任何一個測試案例中，如果下列條件中至少有一個成立，那麼您的解決方案在該測試案例中的得分就是 0 (在 CMS 中報告為 `Output isn't correct`)：

- S 中至少有一個元素無效。
- $Z > 20\,000$ 或 $M > 50$ 。
- 呼叫子程序 `longest_path` 的回傳值不正確。

否則，您在任務 1 的得分計算如下：

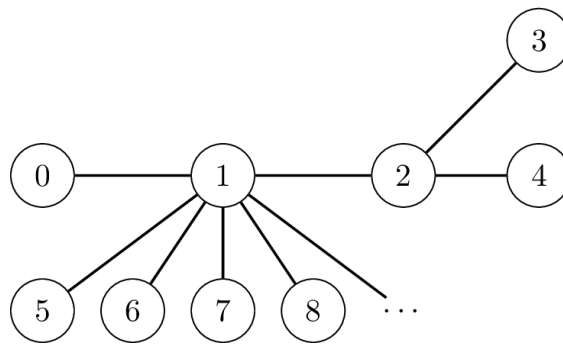
條件	分數
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

您在任務 2 的得分計算如下：

條件	分數
$5 \leq Z \leq 20\,000$ 和 $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ 和 $32 \leq M \leq 50$	40
$Z \leq 4$ and $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ 和 $M \leq 8$	70

範例

讓 $N = 10\,000$. 考慮 $P[1] = 0, P[2] = 1, P[3] = 2, P[4] = 2$ 、和 $P[i] = 1$ 為 $i > 4$.



假設研究團隊的策略是每當 $10 \cdot V + U(U, V)$ 的最大距離因 `send_message` 通話而改變時，就發送訊息。

最初，具有最大距離的地點對是 $(U, V) = (0, 0)$ 。在程式第一次執行時，考慮以下的呼叫順序：

子程序呼叫	(U, V)	返回值 ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

請注意，在所有剩餘的呼叫中， $P[i]$ 的值是 1。這表示最大距離的配對沒有改變、團隊不再傳送任何訊息。

然後，在程式的第二次執行中，進行以下的呼叫：

```
longest_path([0, 10, 20, 30, 0, ...])
```

博物館讀取研究團隊傳送的最後一封訊息，即 $S[3] = 30$ 、並推斷 $(0, 3)$ 是距離最大的一對地點。因此，呼叫返回 $(0, 3)$ 。

請注意，這種方法並不總是能讓博物館正確判斷出具有最大距離的地點對。

樣例評分器

樣例評分器在同一個執行中同時呼叫 `send_message` 和 `longest_path`，這與實際的分級器不同。

輸入格式：

```
N
P[1] P[2] ... P[N-1]
```

輸出格式：

```
S[1] S[2] ... S[N-1]  
U V
```

請注意，您可以使用 N 的任何值來使用樣例分評器。